

Fiche de synthèse structurée MLP et PyTorch

Construction des modèles • Gestion des paramètres • Initialisation • Sauvegarde/chargement • Utilisation du GPU

Cette fiche unifie le contenu des documents fournis en une synthèse pédagogique claire, avec définitions, concepts clés, formules, exemples simples et scripts Python commentés.

1. Vue d'ensemble

En PyTorch, la brique fondamentale de modélisation est la classe **nn.Module**. Un module peut représenter une couche, un bloc de calcul ou un réseau complet. Il reçoit une entrée, calcule une sortie via la méthode **forward**, et peut contenir des paramètres apprenables, des sous-modules et du code Python personnalisé.

- Construire un modèle avec `nn.Sequential` ou une classe personnalisée.
- Inspecter et gérer les paramètres avec `weight`, `bias`, `state_dict()` et `named_parameters()`.
- Initialiser correctement les poids et biais pour stabiliser l'apprentissage.
- Sauvegarder et recharger tenseurs et modèles avec `torch.save` et `torch.load`.
- Utiliser le GPU en gardant modèle et données sur le même device.

2. Définitions essentielles

2.1 Module

Un module est un objet dérivé de `nn.Module`. Il encapsule une transformation de l'entrée vers la sortie et s'intègre automatiquement au calcul des gradients.

2.2 Paramètre

Un paramètre est une variable apprenable du modèle. En pratique, il s'agit le plus souvent des poids et des biais des couches linéaires ou convolutives.

2.3 Gradient

Le gradient mesure la sensibilité de la fonction de perte par rapport à un paramètre. Il est utilisé par l'optimiseur pour mettre à jour les paramètres après `loss.backward()`.

2.4 state_dict

Le `state_dict()` d'un modèle est un dictionnaire qui associe à chaque nom de paramètre son tenseur correspondant. C'est le format standard de sauvegarde des paramètres.

2.5 Device

Un device indique l'emplacement mémoire et matériel d'exécution d'un tenseur ou d'un modèle : CPU ou GPU.

3. Construction d'un MLP en PyTorch

3.1 Avec nn.Sequential

nn.Sequential enchaîne les modules dans l'ordre. Si l'on a $M_1, M_2, \dots, M_k$, alors l'appel `Sequential(M1, M2, ..., Mk)(X)` applique successivement toutes les transformations à X .

Exemple 1 — perceptron multicouche simple

```
import torch
from torch import nn

net = nn.Sequential(
    nn.Linear(256),
    nn.ReLU(),
    nn.Linear(10)
)

X = torch.rand(2, 20)
Y = net(X)
print(Y.shape)  # torch.Size([2, 10])
```

Ici, la première couche produit 256 caractéristiques cachées, ReLU introduit la non-linéarité, puis la dernière couche produit 10 sorties.

3.2 Avec une classe personnalisée

Exemple 2 — MLP défini comme classe

```
import torch
from torch import nn
from torch.nn import functional as F

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(256)
        self.out = nn.Linear(10)

    def forward(self, X):
        H = F.relu(self.hidden(X))
        return self.out(H)
```

Cette approche est plus flexible : elle permet de créer des architectures non linéaires, des branchements, des partages de paramètres ou des opérations spécifiques dans `forward()`.

4. Formules fondamentales

Concept	Formule / explication
Couche linéaire	$y = XW + b$ (ou $y = XW^T + b$ selon la convention)
Couche cachée MLP	$H = \text{ReLU}(XW_1 + b_1)$
Sortie MLP	$Y = HW_2 + b_2$
Activation ReLU	$\text{ReLU}(z) = \max(0, z)$
Modèle paramétré	$f_\theta(x)$, où θ désigne l'ensemble des paramètres

Une bonne lecture conceptuelle consiste à retenir que le réseau transforme progressivement l'entrée en représentation cachée, puis en sortie, en combinant opérations linéaires et fonctions d'activation.

5. Gestion des paramètres

5.1 Accès à une couche et à ses paramètres

```
print(net[2])          # affiche la couche
print(net[2].state_dict()) # affiche poids et biais
```

Dans un réseau séquentiel, on peut accéder à une couche par son indice. Le `state_dict()` de la couche contient généralement `weight` et `bias`.

5.2 Paramètre, valeur numérique, gradient

```
print(net[2].weight)      # objet Parameter
print(net[2].weight.data) # valeurs numériques
print(net[2].weight.grad) # gradient après backward()
```

Avant la rétropropagation, `.grad` vaut en général `None`. Après une propagation avant, le calcul de la perte puis l'appel à `backward()`, le gradient devient disponible.

5.3 Inspection globale

```
for name, param in net.named_parameters():
    print(name, param.shape)
```

`named_parameters()` est très utile pour comprendre la structure du réseau, vérifier les dimensions et déboguer un modèle.

5.4 Partage de paramètres

```
shared = nn.LazyLinear(8)

net = nn.Sequential(
    nn.LazyLinear(8), nn.ReLU(),
    shared, nn.ReLU(),
    shared, nn.ReLU(),
    nn.LazyLinear(1)
)
```

Réutiliser la même couche signifie partager exactement les mêmes poids. Cela réduit le nombre de paramètres, économise de la mémoire et impose une cohérence structurelle entre plusieurs parties du réseau.

6. Initialisation des paramètres

L'initialisation influence fortement la vitesse d'apprentissage, la stabilité numérique et la capacité du modèle à converger.

6.1 Initialisation gaussienne

```
def init_normal(module):
    if isinstance(module, nn.Linear):
        nn.init.normal_(module.weight, mean=0, std=0.01)
        nn.init.zeros_(module.bias)
```

Cette stratégie garde les poids petits au départ, ce qui évite des activations trop grandes dans les premières itérations.

6.2 Initialisation constante

```
def init_constant(module):
    if isinstance(module, nn.Linear):
        nn.init.constant_(module.weight, 1.0)
        nn.init.zeros_(module.bias)
```

Pédagogiquement utile, cette méthode est généralement mauvaise pour l'entraînement réel, car tous les neurones d'une même couche commencent de façon identique : c'est le problème de symétrie.

6.3 Initialisation de Xavier

```
def init_xavier(module):  
    if isinstance(module, nn.Linear):  
        nn.init.xavier_uniform_(module.weight)  
        nn.init.zeros_(module.bias)
```

Xavier cherche à stabiliser la variance des signaux lors de la propagation. C'est une stratégie souvent bien adaptée aux réseaux profonds.

6.4 Initialisation personnalisée

```
def my_init(module):  
    if isinstance(module, nn.Linear):  
        nn.init.uniform_(module.weight, -10, 10)  
        module.weight.data *= (module.weight.data.abs() >= 5)  
        nn.init.zeros_(module.bias)
```

Cette approche montre que PyTorch laisse un contrôle très fin au programmeur : on peut imposer des distributions non standard, de la parcimonie ou des contraintes métier.

7. Sauvegarde et rechargement

7.1 Sauvegarder un tenseur

```
x = torch.arange(4)  
torch.save(x, "x-file")  
x2 = torch.load("x-file")  
print(x2)
```

7.2 Sauvegarder plusieurs objets

```
x = torch.arange(4)  
y = torch.zeros(4)  
  
torch.save([x, y], "x-files")  
x2, y2 = torch.load("x-files")
```

7.3 Sauvegarder un modèle

```
torch.save(net.state_dict(), "mlp.params")
```

La bonne pratique n'est pas de sauvegarder directement l'objet modèle, mais son `state_dict()`, c'est-à-dire uniquement ses paramètres.

7.4 Recharger un modèle

```
clone = MLP()  
clone.load_state_dict(torch.load("mlp.params"))  
clone.eval()
```

Il faut recréer la même architecture avant de charger les paramètres, car `state_dict()` ne contient pas le code Python du modèle.

8. Utilisation du GPU

Le GPU accélère fortement les opérations matricielles massives. Mais une règle ne doit jamais être oubliée : toutes les données impliquées dans une même opération doivent se trouver sur le même device.

Fonctions utilitaires de sélection du device

```
import torch

def cpu():
    return torch.device("cpu")

def gpu(i=0):
    return torch.device(f"cuda:{i}")

def num_gpus():
    return torch.cuda.device_count()

def try_gpu(i=0):
    if num_gpus() >= i + 1:
        return gpu(i)
    return cpu()
```

Exemple minimal de calcul sur GPU

```
device = try_gpu()

X = torch.randn(16, 10, device=device)

net = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
).to(device)

Y = net(X)
print(X.device, Y.device)
```

Pour obtenir de bonnes performances, il faut créer les tenseurs directement sur le bon device, déplacer le modèle une seule fois, et limiter les transferts CPU ↔ GPU.

9. Exemples simples à retenir

- Une couche unique : `nn.LazyLinear(8)` transforme une entrée de taille inconnue vers 8 sorties.
- Un petit réseau : `Linear` → `ReLU` → `Linear` suffit pour un premier MLP de démonstration.
- Un gradient apparaît seulement après calcul de la perte puis `backward()`.
- Une couche partagée réutilise exactement les mêmes paramètres à plusieurs endroits.
- Un modèle et ses données doivent être sur le même device pour fonctionner correctement.

10. Script Python récapitulatif commenté

```

import torch
from torch import nn
from torch.nn import functional as F

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.LazyLinear(8)
        self.output = nn.LazyLinear(1)

    def forward(self, x):
        return self.output(F.relu(self.hidden(x)))

net = MLP()
X = torch.rand(2, 4)
Y = net(X)  # initialise les LazyLinear

def init_xavier(module):
    if isinstance(module, nn.Linear):
        nn.init.xavier_uniform_(module.weight)
        nn.init.zeros_(module.bias)

net.apply(init_xavier)

for name, param in net.named_parameters():
    print(name, param.shape)

loss = Y.sum()
loss.backward()

torch.save(net.state_dict(), "mlp.params")

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
net = net.to(device)
X = X.to(device)
Y = net(X)
print("Device de sortie :", Y.device)

```

Ce script résume l'essentiel : construction du modèle, initialisation, inspection des paramètres, rétropropagation, sauvegarde et exécution sur le bon device.

11. Erreurs fréquentes

- Oublier `super().__init__()` dans une classe héritée de `nn.Module`.
- Stocker les couches dans une simple liste Python au lieu d'utiliser `ModuleList`, `Sequential` ou `add_module()`.
- Confondre un tenseur ordinaire avec un paramètre apprenable.
- Essayer d'inspecter une `LazyLinear` avant le premier passage de données.
- Oublier `eval()` au moment de l'inférence.
- Mélanger CPU et GPU dans une même opération.
- Modifier `.data` sans comprendre les effets possibles sur autograd.

12. Résumé final

En PyTorch, `nn.Module` est l'abstraction centrale. Les paramètres d'un réseau doivent être bien enregistrés, bien initialisés, correctement inspectés et sauvegardés sous forme de `state_dict()`. Le GPU apporte un gain de vitesse important, à condition que les données et le modèle soient placés sur le même device et que les transferts inutiles soient évités. Maîtriser ces mécanismes permet de construire des modèles de deep learning plus clairs, plus robustes et plus faciles à entraîner, déboguer et réutiliser.